

MyOS — Bare-Metal Operating System for Arduino Uno

1. Projekat

MyOS je eksperimentalni, minimalni bare-metal operativni sistem za **Arduino Uno R3** sa **ATmega328P** procesorom.

Cilj projekta je razviti mali operativni sistem potpuno od nule, bez korišćenja Arduino frameworka, Arduino Core biblioteka ili postojećeg RTOS-a.

Projekat treba da služi kao praktičan uvod u:

- bare-metal development
- AVR arhitekturu
- boot proces
- linker scripts
- startup kod
- UART komunikaciju
- interrupt-e
- timer-e
- GPIO
- memory management
- scheduler
- task management
- bootloader development
- komunikacioni protokol između računara i OS-a

2. Ciljni hardver

Primarni hardver:

Arduino Uno R3

Procesor:

ATmega328P

Osnovne karakteristike:

Architecture: AVR 8-bit
CPU: ATmega328P
Clock: 16 MHz
Flash: 32 KB
SRAM: 2 KB
EEPROM: 1 KB

OS mora biti pisan direktno za ATmega328P.

Ne treba koristiti:

Arduino framework
Arduino Core
setup()
loop()
digitalWrite()
Serial
delay()

Umesto toga treba direktno koristiti:

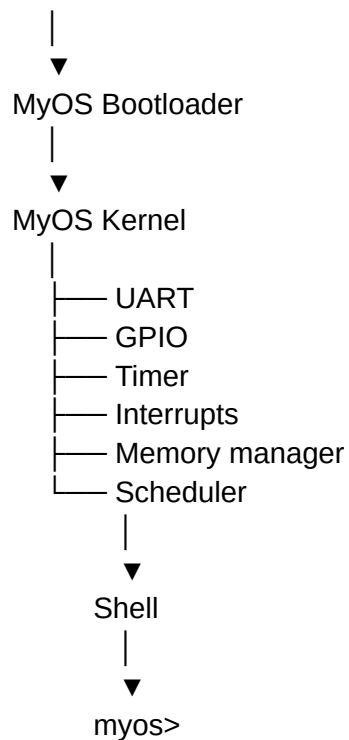
AVR registers
AVR instructions
interrupt vectors
hardware peripherals

3. Krajnji cilj

Konačni MyOS treba da se ponaša kao mali embedded operating system.

Nakon uključivanja:

Arduino Uno
|
▼
RESET



Korisnik treba da može da poveže Arduino sa računarom preko USB-a i da dobije terminal:

```
=====
MyOS v0.1
=====
```

ATmega328P kernel started.

myos>

4. Razvojni princip

Projekat se razvija **inkrementalno**.

Nikada ne implementirati više velikih komponenti odjednom.

Svaki milestone mora:

1. da se kompajlira,
2. da se flashuje,

3. da se testira na fizičkom Arduino Uno uređaju,
4. da se dokumentuje,
5. tek onda prelazi na sledeći milestone.

Kod mora biti što jednostavniji i razumljiviji.

Prioritet je:

Correctness

↓

Understanding

↓

Testability

↓

Performance

↓

Optimization

Ne optimizovati prerano.

5. Development environment

Preporučeni razvojni sistem:

Linux

Potrebni alati:

```
sudo apt install gcc-avr binutils-avr avr-libc avrdude make
```

Proveriti:

```
avr-gcc --version
```

```
avr-objcopy --version
```

```
avrdude --version
```

6. Struktura projekta

Početna struktura:

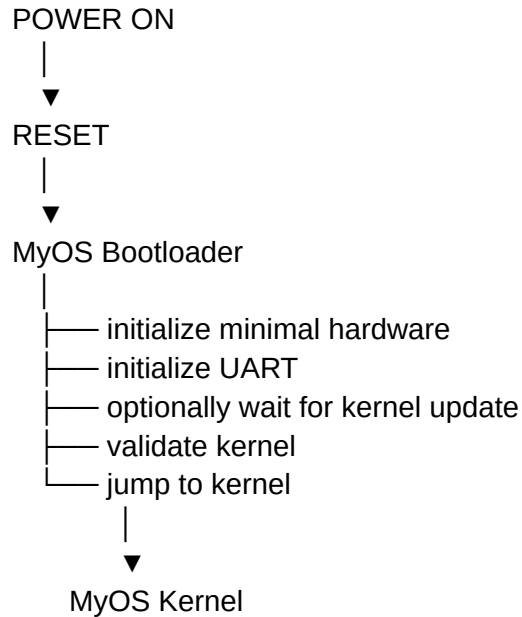
```
myos/
├── boot/
│   └── start.S
├── kernel/
│   ├── kernel.c
│   ├── memory.c
│   ├── memory.h
│   ├── timer.c
│   └── timer.h
├── drivers/
│   ├── uart.c
│   ├── uart.h
│   ├── gpio.c
│   └── gpio.h
├── shell/
│   ├── shell.c
│   └── shell.h
├── include/
├── bootloader/
├── tools/
├── build/
├── linker.ld
├── Makefile
└── README.md
```

7. Boot proces

ATmega328P nakon resetovanja počinje izvršavanje od reset vektora.

U zavisnosti od fuse konfiguracije, reset može biti usmeren na application ili bootloader sekciju.

Konačni boot proces treba da bude:



8. Bootloader vs Kernel

Bootloader i kernel moraju biti odvojene komponente.

Bootloader

Odgovoran za:

- minimalnu hardware inicijalizaciju
- UART komunikaciju
- proveru da li postoji novi kernel
- prijem kernel image-a
- programiranje Flash-a
- proveru integriteta
- pokretanje kernela

Kernel

Odgovoran za:

- hardware abstraction
 - interrupts
 - timers
 - memory management
 - scheduler
 - task management
 - shell
 - sistemske funkcije
-

9. Prva razvojna faza — bez custom bootladera

U prvoj fazi koristiti postojeći Arduino/Optiboot bootloader.

Cilj:

Optiboot

↓

MyOS kernel

Razlog je bezbednost i jednostavnost razvoja.

Ne menjati fuse konfiguraciju niti originalni bootloader dok kernel ne bude funkcionalan.

10. Startup kod

Kernel mora imati sopstveni startup kod.

Primer koncepta:

`.global _start`

`_start:`

`; initialize stack pointer`

```
; initialize CPU state
```

```
rcall kernel_main
```

```
hang:
```

```
    rjmp hang
```

Startup kod mora:

1. podesiti stack pointer,
 2. pripremiti minimalno CPU stanje,
 3. pozvati `kernel_main`,
 4. sprečiti izvršavanje nepoznatog memorijskog prostora ako se kernel vrati.
-

11. Linker script

Napraviti sopstveni `linker.ld`.

Potrebno je definisati:

FLASH

SRAM

ATmega328P:

FLASH = 32 KB

SRAM = 2 KB

Kernel kod treba da bude smešten u Flash.

Globalni podaci i stack treba da budu u SRAM-u.

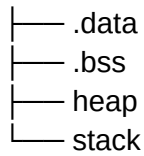
Konačna memorijska organizacija treba jasno da definiše:

Flash

|— kernel

|— bootloader

SRAM



Tačne granice bootloader sekcije definisati tek kada se uvede custom bootloader i izabere konkretna fuse konfiguracija.

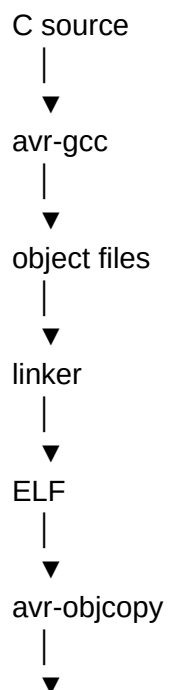
12. Build sistem

Build sistem mora biti nezavisan od Arduino IDE-a.

Koristiti:

avr-gcc
avr-objcopy
avr-ld / linker preko avr-gcc
make

Build pipeline:



HEX

Rezultat:

build/myos.elf
build/myos.hex

13. Milestone 1 — Kernel boot

Prvi cilj:

Power on

↓

Optiboot

↓

MyOS

↓

kernel_main()

Kernel ne mora još ništa da radi.

Uspešan test znači da program stabilno radi beskonačnu petlju.

14. Milestone 2 — UART

Implementirati direktni USART driver.

API:

```
void uart_init(void);  
void uart_putc(char c);  
void uart_puts(const char *str);  
char uart_getc(void);
```

Za početak koristiti:

9600 baud
8 data bits
no parity
1 stop bit

Kasnije se može povećati brzina.

15. Milestone 3 — Prvi kernel output

Kernel nakon bootovanja treba da prikaže:

```
=====
      MyOS v0.1
=====
```

ATmega328P kernel started.

myos>

Ovo je prvi veliki milestone.

Kada ovo radi, imamo:

```
boot
+
kernel
+
UART
```

16. Milestone 4 — UART input

Implementirati primanje karaktera.

Primer:

myos> hello

hello

Za početak echo ponašanje:

received character

↓

uart_getc()

↓

uart_putc()

17. Milestone 5 — Shell

Napraviti minimalni command parser.

Primer:

```
myos> help
myos> info
myos> uptime
myos> mem
myos> gpio
myos> echo
myos> reboot
```

Shell treba da:

1. čita input,
 2. obrađuje backspace,
 3. prepoznaje ENTER,
 4. parsira command,
 5. izvršava odgovarajuću funkciju,
 6. ponovo prikaže prompt.
-

18. Shell komande

help

myos> help

help
info
uptime
mem
gpio
echo
reboot

info

myos> info

OS: MyOS
Version: 0.1
CPU: ATmega328P
Clock: 16 MHz
Flash: 32768 bytes
RAM: 2048 bytes
EEPROM: 1024 bytes

echo

myos> echo hello
hello

19. Milestone 6 — GPIO

Implementirati GPIO driver direktnim pristupom AVR registrima.

API:

```
void gpio_init(void);  
void gpio_set(unsigned char pin);  
void gpio_clear(unsigned char pin);  
unsigned char gpio_read(unsigned char pin);
```

Test:

```
myos> gpio 13 on  
GPIO 13: ON
```

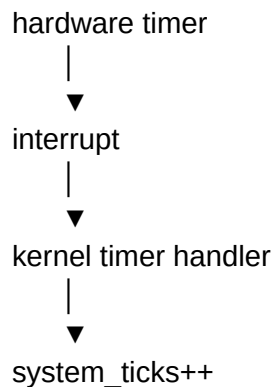
```
myos> gpio 13 off  
GPIO 13: OFF
```

Na standardnom Uno-u digitalni pin 13 koristi PB5.

20. Milestone 7 — Timer

Implementirati jedan hardware timer.

Cilj:



Napraviti:

```
unsigned long timer_get_ticks(void);
```

Shell:

```
myos> uptime
```

```
Ticks: 12345  
Uptime: 12 seconds
```

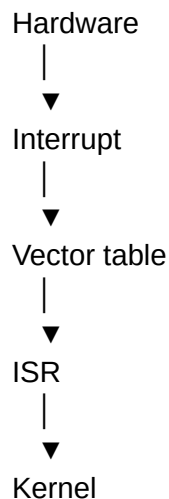
21. Milestone 8 — Interrupt system

Implementirati interrupt infrastrukturu.

Potrebno je:

- razumeti AVR interrupt vector tabelu,
- definisati interrupt handlera,
- omogućiti globalne interrupt-e,
- napisati timer ISR,
- kasnije UART ISR.

Koncept:



22. Milestone 9 — UART interrupts

Prebaciti UART sa polling modela na interrupt-driven model.

Umesto:

```
while (!RX_READY)
    ;
```

koristiti:

UART RX interrupt



ISR



receive buffer



shell

Napraviti circular/ring buffer.

Primer:

UART



RX ISR



Ring Buffer



Shell

23. Milestone 10 — Memory manager

ATmega328P ima samo 2 KB SRAM-a, pa memory manager mora biti jednostavan.

Napraviti:

```
void *kmalloc(unsigned int size);
```

```
void kfree(void *ptr);
```

Za početak koristiti veoma jednostavan allocator.

Shell:

```
myos> mem
```


RAM total: 2048
Used: 320
Free: 1728

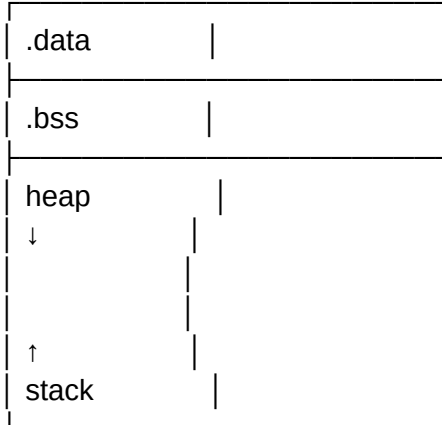
Obavezno voditi računa o:

- stack-u,
 - heap-u,
 - fragmentaciji,
 - overflow-u,
 - NULL rezultatima.
-

24. Memorijska organizacija

Poželjno:

SRAM
0x0100



Heap i stack ne smeju da se sudare.

Kernel treba da ima mogućnost da detektuje približavanje stack/heap kolizije.

25. Milestone 11 — Tasks

Uvesti koncept task-a.

Struktura:

```
typedef struct
{
    unsigned char id;
    unsigned char state;
    void (*entry)(void);
} task_t;
```

Početno podržati nekoliko taskova.

Primer:

Task 1 → shell

Task 2 → LED

Task 3 → system service

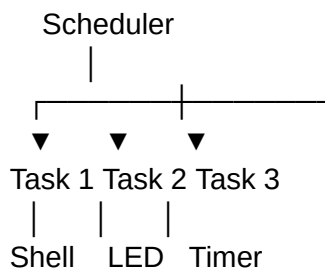
26. Milestone 12 — Scheduler

Prvo implementirati **cooperative scheduler**.

API:

```
void scheduler_init(void);
void scheduler_run(void);
void task_yield(void);
```

Koncept:



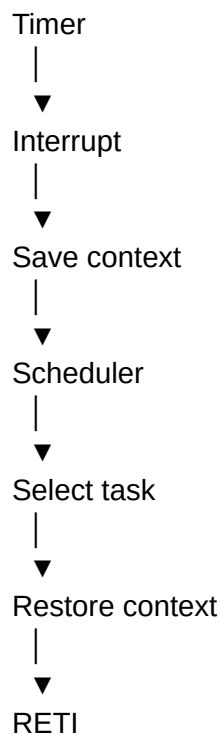
Task mora eksplicitno da prepusti CPU:

```
task_yield();
```

27. Milestone 13 — Preemptive scheduler

Tek nakon stabilnog cooperative schedulera pokušati preemptive scheduling.

Timer interrupt:



Potrebno implementirati:

- register context saving,
- stack switching,
- task stack,
- task state,
- scheduler,
- context restore.

Ovo je jedan od najvažnijih OS-development delova projekta.

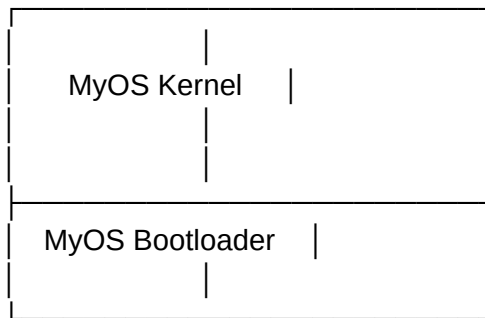
28. Milestone 14 — Custom Bootloader

Tek sada ukloniti zavisnost od Optiboot-a.

Konačna struktura Flash-a:

FLASH

0x0000



Fuse konfiguraciju i konkretne adrese bootloader sekcije pažljivo definisati prema datasheet-u i izabranoj konfiguraciji.

29. Custom bootloader funkcije

Bootloader treba da:

RESET



UART INIT



WAIT FOR HOST

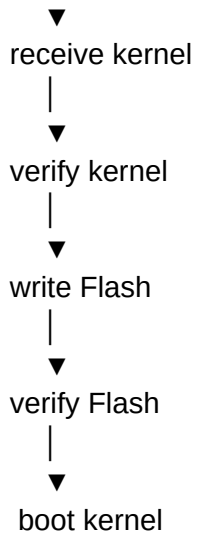


timeout



update request





30. Custom update protocol

PC tool i bootloader treba da koriste sopstveni protokol.

Primer:

HOST → HELLO

BOOT → READY

HOST → KERNEL_INFO

BOOT → OK

HOST → PAGE

BOOT → ACK

HOST → PAGE

BOOT → ACK

...

HOST → VERIFY

BOOT → OK

HOST → BOOT

Svaki paket treba da ima:

MAGIC
TYPE
LENGTH
SEQUENCE
PAYLOAD
CRC

Primer:

```
+-----+-----+-----+-----+-----+-----+  
| MAGIC | TYPE | LENGTH | SEQUENCE | PAYLOAD | CRC |  
+-----+-----+-----+-----+-----+-----+
```

31. Kernel Flashing Tool

Napraviti PC alat:

tools/myos-upload

Na primer:

./myos-upload build/kernel.hex

Alat treba da:

1. otvori serial port,
 2. pronade bootloader,
 3. izvrši handshake,
 4. pročita kernel image,
 5. podeli ga na Flash page-ove,
 6. šalje page-ove,
 7. čeka ACK,
 8. proverava CRC,
 9. traži boot.
-

32. Flash programming

Bootloader treba da koristi ATmega328P self-programming mogućnost za upisivanje Flash memorije.

Ne programirati Flash byte-po-byte.

Koristiti page-based pristup.

Koncept:

kernel image



page 0

page 1

page 2

page 3

...



Flash

33. Reboot

Implementirati:

myos> reboot

Reboot može koristiti watchdog mehanizam.

Koncept:

command



watchdog reset



bootloader



kernel

34. EEPROM konfiguracija

EEPROM može da se koristi za persistent konfiguraciju.

Na primer:

EEPROM
├── magic
├── OS version
├── configuration
└── checksum

Moguća upotreba:

boot configuration
device name
shell settings
boot flags

35. Watchdog

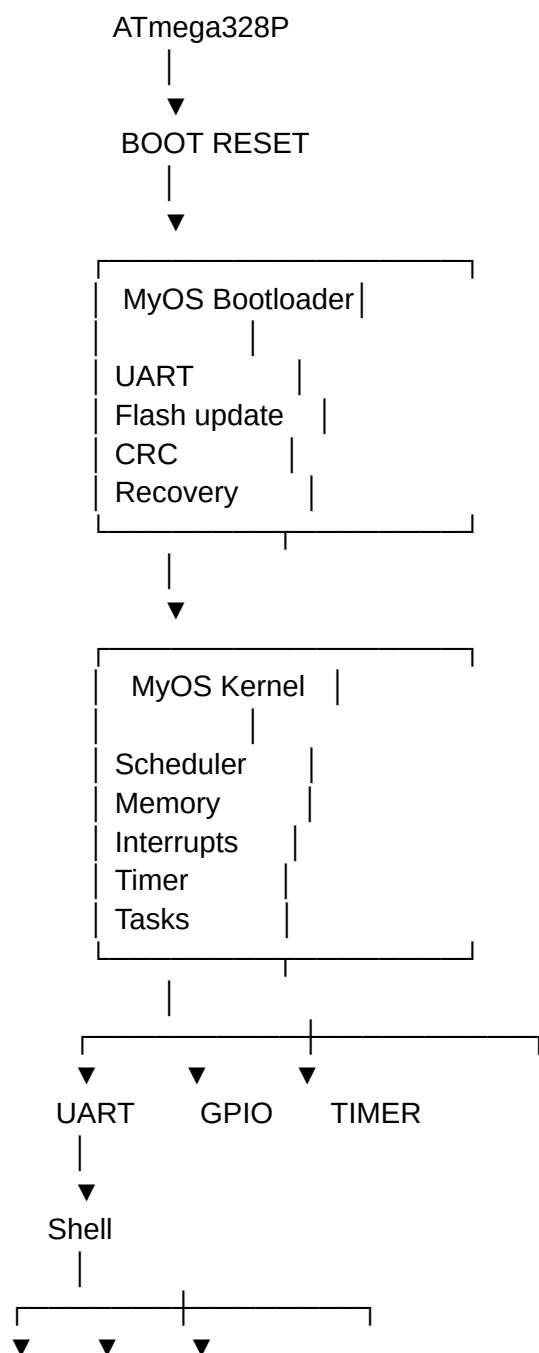
Implementirati watchdog kao sigurnosni mehanizam.

Koncept:

Kernel
├── periodically reset watchdog
└── if kernel hangs
 ▼
 watchdog reset
 ▼
 bootloader

Kasnije watchdog može biti korišćen za recovery.

36. Konačna arhitektura MyOS-a



help info mem

37. Final shell

Cilj je da sistem na kraju podržava:

```
myos> help
```

Available commands:

help	Show help
info	System information
mem	Memory information
uptime	System uptime
gpio	GPIO control
tasks	List tasks
echo	Print text
reboot	Restart system
version	OS version

Primer:

```
myos> info
```

MyOS 1.0

CPU: ATmega328P
Clock: 16 MHz
Flash: 32768 bytes
SRAM: 2048 bytes
EEPROM: 1024 bytes

Kernel: OK
Timer: OK
UART: OK
Scheduler: OK

38. Testing strategy

Svaka komponenta mora imati test.

Boot test

RESET → kernel_main()

UART test

TX → RX → echo

GPIO test

GPIO 13 ON/OFF

Timer test

ticks increment

Interrupt test

timer interrupt count

Memory test

kmalloc

kfree

Scheduler test

Task A

Task B

Task C

Bootloader test

PC → kernel → Flash → reboot → kernel

39. Debugging

Kada god je moguće, debugging raditi preko UART-a.

Primer:

```
[BOOT] starting  
[BOOT] UART OK  
[BOOT] kernel found  
[BOOT] jumping to kernel
```

```
[KERNEL] start  
[KERNEL] memory OK  
[KERNEL] timer OK  
[KERNEL] scheduler OK
```

U finalnom buildu omogućiti:

DEBUG=0

kako bi debug output mogao da se ukloni.

40. Dokumentacija

Svaka komponenta treba da ima kratku dokumentaciju.

Na primer:

```
docs/  
├── boot.md  
├── memory.md  
├── uart.md  
└── interrupts.md
```

```
|— timer.md
|— scheduler.md
|— bootloader.md
|— protocol.md
```

Dokumentacija treba da objašnjava:

- kako hardware radi,
 - koji registri se koriste,
 - kako kod koristi hardware,
 - zašto je određena implementacija izabrana.
-

41. Važno razvojno pravilo

Ne pokušavati da napraviš "veliki OS" odmah.

Redosled mora biti:

```
Boot
↓
Kernel
↓
UART
↓
Shell
↓
GPIO
↓
Timer
↓
Interrupts
↓
Memory
↓
Tasks
↓
Scheduler
↓
Bootloader
↓
```

Flash update



EEPROM



Watchdog

Svaki korak mora raditi pre nego što se pređe na sledeći.

42. Prvi konkretan milestone

Prvi cilj projekta je:

Arduino Uno



existing bootloader



MyOS kernel



UART initialization



Terminal mora prikazati:

```
=====
MyOS v0.1
=====
```

ATmega328P kernel started.

myos>

I zatim:

```
myos> hello
hello
```

Kada ovo radi, **Milestone 1 je završen.**

Tek nakon toga počinje implementacija GPIO-a, timer-a i interrupt sistema.

43. Tehnička dokumentacija

Primarni izvor informacija za implementaciju treba da bude **ATmega328P datasheet**.

Posebno proučiti:

Architecture
Memory
AVR CPU Core
Interrupts
I/O Ports
16-bit Timer/Counter
8-bit Timer/Counter
USART
SPI
TWI
EEPROM
Watchdog
Boot Loader Support
Self-Programming
Fuse Bits
Memory Programming

Kod treba pisati na osnovu dokumentacije procesora, a ne na osnovu pretpostavki izvedenih iz Arduino API-ja.

44. Definition of Done

MyOS prva kompletna verzija smatra se završenom kada:

- sopstveni kernel bootuje

- UART radi bez Arduino API-ja
- postoji shell
- GPIO može da se kontroliše
- timer radi
- interrupt sistem radi
- memory allocator radi
- postoje taskovi
- scheduler radi
- custom bootloader radi
- kernel može da se uploaduje preko custom protokola
- Flash image se validira
- EEPROM konfiguracija radi
- watchdog recovery radi
- dokumentacija postoji
- projekat se kompletno build-uje jednom **make** komandom

Konačni rezultat treba da bude **mali, razumljiv, potpuno bare-metal operativni sistem za ATmega328P**, koji se može bootovati, kontrolisati preko terminala, izvršavati osnovne sistemske funkcije i samostalno ažurirati svoj kernel.